

Automation and GUI Testing Tools

UPL - 2026

Why GUI Testing Needs Automation

- GUI testing often involves repetitive work. Testers may need to rerun the same scenarios after every software update, across multiple browsers, or under many input conditions.
- Manual testing is still useful, but it becomes expensive and time-consuming when repetition is high.
- Automation helps because it can:
 - repeat the same steps consistently
 - support regression testing
 - reduce human effort for repetitive tasks
 - scale testing across browsers and environments

Manual Testing vs Automated Testing

- Manual GUI testing means a human tester directly interacts with the application and checks whether the behavior is correct. This is useful for exploration, visual inspection, and usability judgment.
- Automated GUI testing means a script or tool performs actions on the interface automatically and verifies expected outcomes.

Manual testing	Automated testing
flexible	faster for repeated execution
useful for exploratory testing	more consistent
useful for usability-related observation	useful for regression testing
slower for repetition	requires setup and maintenance
more dependent on human consistency	can become fragile if the GUI changes

Capture-Replay in GUI Testing

- Capture-replay, also known as record-and-replay, is a technique where a tester performs GUI actions once and a tool records those actions. Later, the same action sequence can be replayed automatically.
- Example, a recorded test may include:
 - open the login page
 - click the username field
 - type a username
 - click the password field
 - type a password
 - click login

Advantages of Capture-Replay

- Strengths
 - easy to understand
 - helpful for repetitive workflows
 - useful as an entry point into automation
- Limitations
 - recorded scripts may break if the interface changes
 - tests can become too literal and hard to maintain
 - less suitable for highly dynamic interfaces

GUI Ripping in GUI Testing

- GUI ripping is the process of automatically exploring an application to discover its interface structure.
- A ripper may detect:
 - windows or screens
 - buttons, fields, menus, and dialogs
 - possible user events
 - relationships among components
- Unlike capture-replay, which follows an already demonstrated path, GUI ripping helps a tool inspect the interface on its own. This can support model construction and test generation.

General Workflow of Automated GUI Testing

- A typical automated GUI testing workflow includes:
 - discovering the GUI
 - identifying interface elements and actions
 - building a representation or model
 - generating test cases
 - executing those test cases
 - collecting and analyzing the results
- The event-flow model can become the basis for systematic automated testing.
- Some modern tools focus more on scripting and execution than on explicit model construction.

Popular Tools for Web GUI Automation

- Modern GUI automation tools in practice are often web automation tools.
- Example of GUI automation tools:
 - Selenium
 - Playwright
 - Puppeteer
 - Cypress

Selenium

- <https://www.selenium.dev/>

Playwright

- <https://playwright.dev/>

Puppeteer

- <https://pptr.dev/>

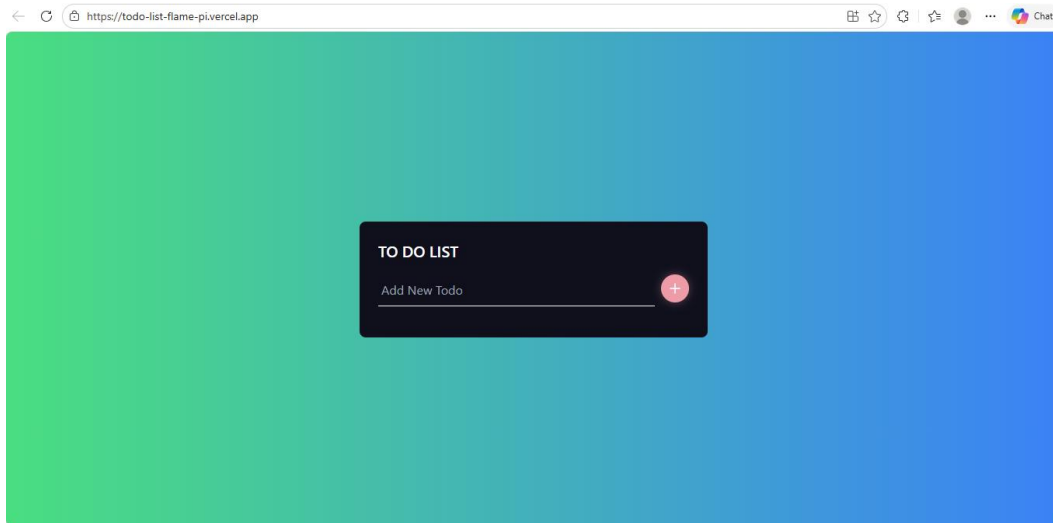
Cypress

- <https://www.cypress.io/>

Discussion: Choosing the Right Tool

Case Study:

<https://todo-list-flame-pi.vercel.app/>



Questions:

- What parts should be tested manually?
- What parts should be automated?
- Which tool would be most suitable: Selenium, Playwright, Puppeteer, or Cypress?
- Why that tool is the best fit for the scenario?